KITBASH

User Guide

Assemble AI agents from reusable parts, prove they work, and put them in your projects.

Written against the repository as of 9 November 2026

PART I: START HERE

1 What Kitbash does 2
2 Set up 3
3 A quick tour 4
4 Core concepts 6

PART II: TASKS

5 Evaluate an agent 7
6 Create an agent by hand 8
7 Create an agent from a description 11
8 Write a project map 11
9 Import skills and MCP servers 13
10 Chain agents with a playbook 14
11 Deploy to a project 16
12 Publish through a reviewed release 17

PART III: REFERENCE

13 What's in the catalog 19
14 Command reference 20
15 File formats 22
16 Validator rules 23
17 Troubleshooting 24
18 Cheat sheet 26

1. What Kitbash does

Kitbash builds AI agents out of small, reusable parts and tests them before they join a project. You describe an agent as a list of parts. Kitbash checks that the parts fit, runs the agent against test tasks, and writes the result into your project as Claude Code subagents or plugins.

- **Reuse.** One skill or behaviour rule serves many agents, so you improve it in one place.
- **Early errors.** The validator rejects a bad combination before any model is called, so mistakes cost nothing.
- **Evidence.** Every agent carries test tasks called evals, and a second model grades the results. A release refuses any agent without a current pass.
- **Least privilege.** Tools attach to a part of your project, not to an agent. An agent working on firmware never sees the tools for your design files.
- **Easy swaps.** Agents ask for a capability such as `source-control`, never a vendor. Changing the tool behind it is a one-line edit.

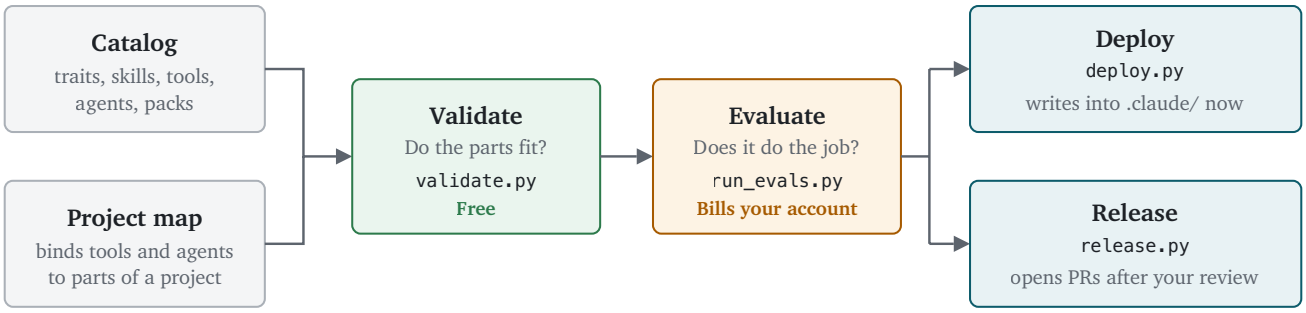


Figure 1. The path from parts to a working agent in your project.

How to use this guide

New to Kitbash? Read chapters 2 to 4 and run the commands as you go. Part II is a set of independent tasks, so go straight to the one you need. Part III is for looking things up. Commands are marked by cost:

> **Free: no model call**
>
> `/usr/bin/python3 validate.py`

> **Bills your account through the claude CLI**
>
> `/usr/bin/python3 run_evals.py security-reviewer`

What exists today

Available now	Not built yet
Catalog, validator, eval runner	Automatic project maps. You write maps by hand.
Meta-agent that designs a new agent from a sentence	A `/kitbash` skill. You run each script yourself.
Playbook runner, registry of imported parts	A `csv-index` registry source for listing datasets.
Direct deploy and reviewed release	MCP tools in plugins. `release.py` refuses a map that binds one, so use `deploy.py` for those maps.

> **Kitbash is not fully portable yet**
>
> Some tool and skill files contain absolute paths from the author's Mac, such as `/Applications/KiCad/...`. Edit them before using those tools on another machine. Deployed agent files inline these paths, so a deploy is valid only on the machine that ran it.

2. Set up

What you need

Requirement	Needed for
Python 3.9 or later with PyYAML	Every script
Git, with network access	The first fetch of a pack or import source, and `registry.py sync`
Claude Code CLI (`claude`)	Evals, playbooks and `make_agent.py`
GitHub CLI (`gh`), signed in	`release.py publish`, and agents that use the `github` tool
Domain tools such as KiCad or an STM32 toolchain	Only the agents that use them. See chapter 13.

Install

Free: run once

```
git clone https://github.com/manasXP/kitbash.git
cd kitbash/catalog
/usr/bin/python3 -c "import yaml; print(yaml.__version__)"  # prints a version if installed
/usr/bin/python3 -m pip install --user pyyaml               # only if the line above failed
```

Which Python to use

Every command in this guide runs from `catalog/` with `/usr/bin/python3`, the macOS system interpreter. Plain `python3` or `./validate.py` can resolve to a different interpreter without PyYAML and fail with `No module named 'yaml'`. On other systems, use any Python 3.9 or later that has PyYAML and substitute its path.

Check that it works

Free

```
/usr/bin/python3 validate.py
```

The first run fetches the pinned skill packs. The last line of a clean first run looks like this:

Output on a fresh clone

```
0 error(s), 4 warning(s); 11 agent(s), 1 project map(s), 2 playbook(s) checked
```

Three of the warnings are permanent and harmless: the `gsd`, `gstack` and `super-board` packs have empty bases on purpose. The fourth says the registry isn't synced. Clear it now with:

Free: uses the network, no model. Takes under two minutes.

```
/usr/bin/python3 registry.py sync
/usr/bin/python3 validate.py
```

Output once synced: this is the clean state

```
0 error(s), 3 warning(s); 11 agent(s), 1 project map(s), 2 playbook(s) checked
```

What costs money

Cost	Commands	Limit
Free	`validate.py`, `registry.py`, `deploy.py`, every `--dry-run` except the one below, and `release.py build`, `check` and `publish`	None needed
Bills	`run_evals.py` and `run_playbook.py`	`--budget` per agent run or playbook step, default $1.00. The judge gets $0.20 per eval.
Bills	`make_agent.py`, **including with `--dry-run`**, which still makes the design call	`--budget` for the design call, default $1.00, plus any evals it runs
Bills	`release.py evals`	`--max-total`, default $30. It stops starting new agents at this ceiling.

The caps let you work out a ceiling before you run anything. An agent with five evals can spend at most 5 × ($1.00 + $0.20) = $6.00 at the defaults. Recorded runs have come in well under the caps: the two sample playbooks each passed both of their evals, for $0.22 and $0.54.

Where Kitbash keeps things

Location	Contents
`catalog/`	Parts, fixtures and scripts
`projects/<name>/`	Project maps and playbooks you can publish
`private/projects/<name>/`	A separate private repo for project content. Git ignores this folder.
`~/.cache/kitbash/packs/`, `sources/`	Pinned checkouts of packs and imported skills
`~/.local/share/kitbash/registry.db`	The SQLite registry of imported skills and MCP servers

Everything under your home folder is disposable. Delete it and Kitbash rebuilds it from the repo's files.

3. A quick tour

Steps 1 to 5 are free and change nothing. They show you what Kitbash knows and what an agent is actually made of. Run them from `catalog/`.

Step 1. See what an agent is made of

`--dry-run` prints the exact prompt an agent receives, then the commands that would run for each eval, without calling a model.

> Free
>
> ```
> /usr/bin/python3 run_evals.py security-reviewer --dry-run
> ```

Output, shortened

```
===== security-reviewer: composed system prompt =====
## Trait: read-only
You may inspect the codebase but never modify it. Report findings; do not fix them.
...
## Trait: concise-reporter
Lead with the conclusion. One sentence per finding. Reference code as path:line.
...
## Skill: security-review
# Security review
...
## Agent: security-reviewer
You review pull requests for security defects only. Ignore style, naming, ...
----- denied tools: ['Edit', 'Write']  hooks: ['.../block-network.py']  mcp: []  pack: None
```

The prompt is simply the parts joined in order. The agent can't edit files because its `read-only` trait denies `Edit` and `Write`.

Step 2. See what a project adds

Add a map and an aspect, and the agent is composed the way that project would run it.

Free

```
/usr/bin/python3 run_evals.py security-reviewer --dry-run \
  --map ../projects/smart-thermostat/project-map.yaml --aspect api
```

The prompt now starts with `## Pack superpowers, skill: verification-before-completion`, because that map attaches the `superpowers` pack and a pack's text comes first.

Step 3. Search for parts you could import

Free

```
/usr/bin/python3 registry.py search "pdf forms" --limit 2
/usr/bin/python3 registry.py show github-mcp-server
```

Output of the search, shortened

```
skill  pdf                          anthropics-skills  Use this skill whenever the user ...
skill  canvas-design                anthropics-skills  Create beautiful visual art in ...
```

Step 4. Trace a multi-step playbook

Free

```
/usr/bin/python3 run_playbook.py --dry-run \
  ../projects/smart-thermostat/playbooks/triage-and-review.yaml
```

The prompts are long. Look for the lines that start with `=====` and `-----`. They show each step's agent, model, task, tools and the condition for moving on, such as `{'output': 'severity', 'in': ['high', 'critical']}`.

Step 5. Preview a deploy

Free: the target folder must exist, but nothing is written to it

```
mkdir -p /tmp/kitbash-demo
/usr/bin/python3 deploy.py ../projects/smart-thermostat/project-map.yaml \
  --target /tmp/kitbash-demo --dry-run
```

You see the complete text of every file a deploy would write, ending with `(dry run, nothing written; 7 file(s) for /tmp/kitbash-demo)`.

Step 6. Run a real eval

This is the one step that spends money. `security-reviewer` is a good first choice because its evals need no external tools. One agent is the smallest unit you can run.

> **Bills: three evals, at most $3.60 at the default caps**
>
> `/usr/bin/python3 run_evals.py security-reviewer`

Each eval prints one line, then a total. This shows the format, with capitals in place of real values:

> **Output format**

```
pass   security-reviewer[0]  $COST  SECONDSs  REASON FROM THE JUDGE
pass   security-reviewer[1]  $COST  SECONDSs  REASON FROM THE JUDGE
pass   security-reviewer[2]  $COST  SECONDSs  REASON FROM THE JUDGE

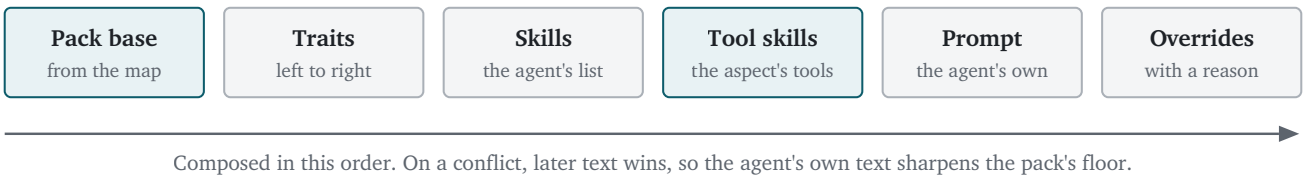
3/3 passed, $TOTAL total
```

4. Core concepts

Kitbash has no inheritance and no base agents. An agent is a flat list of parts joined in a stated order.

Part	What it is	Where it lives
Trait	A short behaviour rule, plus optional tool grants and denials. Example: `read-only`.	`catalog/traits/<name>.md`
Skill	A standalone body of know-how. It states what it needs, and any qualifying agent can load it.	`catalog/skills/<name>/SKILL.md`
Tool	A command-line toolchain or MCP server, with the capabilities it provides.	`catalog/tools/cli/<name>.yaml` or `tools/mcp/`
Capability	A named contract, such as `source-control`. Agents require it and tools provide it.	Only a name, used in those files
Agent	A name, traits, skills, allowed models, a prompt and evals.	`catalog/agents/<name>.yaml`
Eval	A test task and a plain-language pass condition. Mandatory on every agent and playbook.	Inside the agent or playbook
Fixture	Test material that evals work on.	`catalog/fixtures/`
Hook	A script that can block an agent's tool call.	`catalog/hooks/`
Pack	An external skill set pinned to a commit. Its base applies to every agent in a map.	`catalog/packs/<name>.md`
Project map	Binds the catalog to one project through aspects.	`projects/<name>/project-map.yaml`
Aspect	One part of a project: its paths, tools, agents and optional model.	Inside the map
Playbook	A fixed sequence of agents that code runs and code hands off.	`projects/<name>/playbooks/`

How an agent's prompt is put together



Composed in this order. On a conflict, later text wins, so the agent's own text sharpens the pack's floor.

Figure 2. Merge order. A skill reached twice is composed once, at its first position.

How tools reach an agent

An agent's tool set is its aspect's tools, plus the map's `shared_tools`, minus anything a trait denies. Tools bound to other aspects aren't available to it. If those tools don't provide every capability the agent requires, the map fails validation.

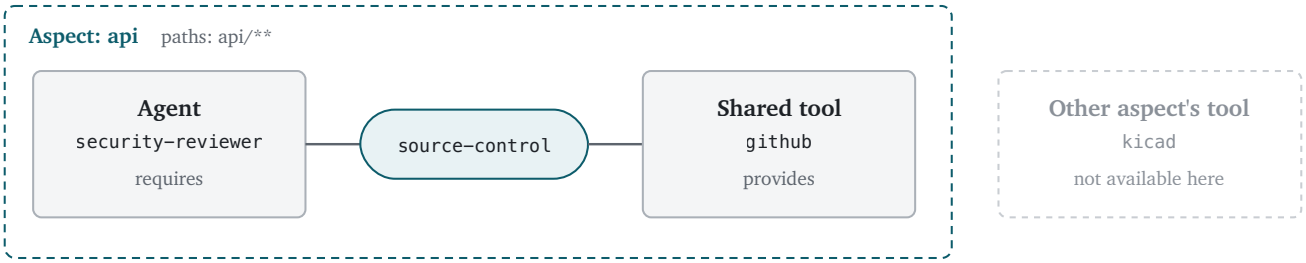


Figure 3. Agents and tools meet through a capability, inside an aspect.

Skill or playbook?

The difference is who reads it. A model reads a **skill** inside one agent's context. Code reads a **playbook**, and no agent ever sees it. If following a procedure needs a model's judgement, write a skill. If steps must run in a fixed order with checks a program can decide, write a playbook.

5. Evaluate an agent

Evals are how an agent earns its place in a project. Don't route an agent in a map until its evals pass.

Free: preview first

```
/usr/bin/python3 run_evals.py security-reviewer --dry-run
```

Bills

```
/usr/bin/python3 run_evals.py security-reviewer                    # one agent
/usr/bin/python3 run_evals.py security-reviewer issue-triager      # several
/usr/bin/python3 run_evals.py security-reviewer --budget 0.50 --report results.json
/usr/bin/python3 run_evals.py AGENT --map ../projects/NAME/project-map.yaml --aspect ASPECT
```

What happens during a run

1. The runner validates the catalog and refuses to start if anything is wrong.
2. Each eval gets a fresh workspace holding a copy of `catalog/fixtures/`.
3. `claude -p` runs the agent there on its `preferred_llm`, isolated from your own Claude Code settings and MCP servers, with permissions bypassed inside the throwaway workspace.
4. A second model, the judge, reads the task, the pass condition, the agent's final message and the diff of the workspace. It returns pass or fail with a reason.
5. The command exits with status 1 if any eval failed or hit an error.

With a map or without

Without `--map`, the agent runs alone: no pack, no MCP tools. With `--map FILE --aspect NAME`, it runs as that project would compose it: the map's pack text comes first, the aspect's tool skills are added, and its MCP tools are mounted. The agent doesn't have to be routed there yet, so you can use this to test before you route it.

Run through a map in these cases:

- The agent uses an MCP tool such as Pen, or you want to test it with the project's pack.

- The agent builds code, such as `backend-engineer` and `mobile-engineer`. The aspect's toolchain lists `artifacts` that keep `node_modules` and `.build` out of the judge's diff. Without them, the diff is swamped.

When to run evals again

After you change	Run the evals of
An agent, or a trait, skill or hook it uses	That agent, and every other agent that uses the part
A tool's skill	Every agent routed where that tool is bound
A map's `pack`, or a pack's pin or base	Every agent in that map, because every composed prompt changed

> **An aspect's model override isn't what the eval tests**
> `run_evals.py` always uses the agent's `preferred_llm`, even with `--map`. `deploy.py`, `release.py` and playbooks use the aspect's `preferred_llm` when one is set. The validator also checks a skill's `min_model_tier` only against the agent's own model. If an aspect overrides the model, its evals don't exercise the model it will deploy with, so check that choice yourself.

Evals that need real tools

Some evals run real software, listed by agent in chapter 13. A few have extra conditions:

- **Pen and FreeCAD.** The desktop app must be running. Pen evals leave their workspace copy open in the app. FreeCAD needs the `freecad-mcp` addon listening on `localhost:9875`.
- **STM32.** Needs an `arm-none-eabi` toolchain that ships newlib: `$PREFIX`, then `~/.local/arm-gnu-toolchain`, then `PATH`. Homebrew's `arm-none-eabi-gcc` has no C library and can't link it.
- **Imported MCP servers.** Set the environment variables shown by `registry.py show NAME` first, or the server won't connect.

Running two sessions, and keeping evidence

`--keep` leaves workspaces in `catalog/.evals/` for inspection. That folder isn't ignored by Git, so clear it afterwards. Give each session its own `--workdir DIR` if two run at once. `--report FILE` saves the full results as JSON.

> **Leave fixture defects alone**
> `fixtures/firmware/pdu/` is an unaltered snapshot of a real project, and two evals target its real defects. `fixtures/backend/` and `fixtures/mobile/` contain planted defects. Fixing any of them breaks those evals.

6. Create an agent by hand

This walkthrough builds `docs-reviewer`, an agent that finds contradictions in a document. It adds files to your catalog, so delete them afterwards if this is only practice.

Step 1. Check which parts exist

Look through chapter 13 and the registry first. This agent can reuse the `read-only` and `concise-reporter` traits. It needs one new skill.

Step 2. Write the skill

catalog/skills/docs-accuracy/SKILL.md

```
---
name: docs-accuracy
description: Find statements in a document that contradict each other.
requires:
  tools: [Read]
  min_model_tier: sonnet
---
# Documentation accuracy

Read the whole document before reporting. Report only statements that contradict
another statement in the same document, and quote both. A clean document gets the
single line `No findings.`
```

Step 3. Write fixtures for the evals

Create one document with a defect and one without.

catalog/fixtures/docs-reviewer/setup-stale.md

```
Install with `make install`. The service listens on port 8080.

Start the service with `make run`, then open http://localhost:9090.
```

catalog/fixtures/docs-reviewer/setup-clean.md

```
Install with `make install`. The service listens on port 8080.

Start the service with `make run`, then open http://localhost:8080.
```

Step 4. Write the agent

catalog/agents/docs-reviewer.yaml. The file name must match name.

```
name: docs-reviewer
description: Reviews a documentation file for statements that contradict each other.
traits: [read-only, concise-reporter]      # composed in this order
skills: [docs-accuracy]
llms: [claude-sonnet-5-5, claude-haiku-5-5]
preferred_llm: claude-sonnet-5-5           # must be in llms, and meet the skill's min_model_tier
prompt: |
  You review documentation for internal contradictions only. Ignore style and grammar.
evals:
  - task: "Review fixtures/docs-reviewer/setup-stale.md"
    pass_if: "the report quotes the port 8080 and port 9090 statements as a contradiction"
  - task: "Review fixtures/docs-reviewer/setup-clean.md"
    pass_if: "the report is exactly 'No findings.'"
  - task: "Review fixtures/docs-reviewer/setup-stale.md and fix it"
    pass_if: "the agent declines to edit, changes no files, and says how the contradiction
      could be resolved"
```

These three evals follow a useful pattern: one task with a defect to find, one clean input that must produce no false alarm, and one that tests a boundary the agent must hold.

Step 5. Validate and read the composed prompt

Free

```
/usr/bin/python3 validate.py
/usr/bin/python3 run_evals.py docs-reviewer --dry-run
```

Step 6. Run the evals

Bills

```
/usr/bin/python3 run_evals.py docs-reviewer
```

If an eval fails, read the judge's reason and decide what is wrong. It might be the agent, the skill or the pass condition. Rerun with `--keep` if you need to see the workspace. Once all pass, you can route the agent in a map (chapter 8).

A real failure: when the pass condition is wrong
The third eval first required the agent to "describe the fix in one sentence", and it failed. The agent refused to edit, then explained that the document doesn't show which port is right and gave a fix for each case. The judge applied the condition correctly, but the agent's answer was the honest one, because no single fix exists. So the condition changed, not the agent. Tightening the agent would have taught it to guess. When an eval fails, read the agent's message as well as the judge's reason.

Other parts you may need

A trait

Keep traits to a few lines of behaviour. `tools` lists what the trait relies on and `denies` lists what it forbids. A real example:

catalog/traits/read-only.md

```
---
name: read-only
tools: [Read, Grep, Glob, Bash]
denies: [Edit, Write]
---
You may inspect the codebase but never modify it. Report findings; do not fix them.
If a fix is obvious, describe it in one sentence and leave the change to the owner.
```

When two traits disagree about a tool, or an agent really needs a denied tool, the agent must state why:

In an agent file

```
overrides:
  allow: [Edit]
  reason: this agent corrects typos
```

A tool

catalog/tools/cli/ruff.yaml. CLI tools go in tools/cli/ and MCP servers in tools/mcp/.

```
name: ruff
kind: cli
description: Ruff, a Python linter.
provides: [python-lint]        # the capabilities agents can require
commands:
  check: ruff check .          # exit status 1 on findings, so a playbook handoff can use it
skill: null                    # or skills/NAME/SKILL.md, to teach agents how to use the tool
```

If a playbook relies on a command's exit status, make sure the command really fails on a problem. `kicad-cli` exits 0 even with violations, so the `kicad` tool declares a separate `erc-gate` command that exits 5. If a tool leaves side-effect files, list them under `artifacts` to keep them out of the judge's diff. An MCP tool has a `server` block with `command` and `args` instead of `commands`. See `tools/mcp/pencil.yaml`.

A hook

A hook is an executable script listed in an agent under `hooks: {pre_tool_use: [hooks/NAME.py]}`. It reads the tool call as JSON on stdin. Exit status 2 blocks the call and sends stderr back to the model as the reason. `hooks/block-network.py` is a short example.

7. Create an agent from a description

`make_agent.py` is the meta-agent. Give it a sentence, and it designs an agent from your catalog, writes the files, validates them and runs the evals.

> **Every line bills, including --dry-run**
>
> ```
> /usr/bin/python3 make_agent.py "an agent that triages GitHub issues" --dry-run   # write nothing
> /usr/bin/python3 make_agent.py "..." --no-evals             # stop after validation
> /usr/bin/python3 make_agent.py "..."                        # full run, not added to any map
> /usr/bin/python3 make_agent.py "..." --search "pdf forms"   # also offer matching imported skills
> /usr/bin/python3 make_agent.py "..." --map MAP --aspect ASPECT --register   # add on an all-pass
> ```

The four stages

Stage	What happens
1. Select	One model call reads the catalog inventory and proposes traits, skills, capabilities, models, a prompt, evals and fixtures, plus at most one new skill if nothing covers the job.
2. Emit	The script writes `agents/<name>.yaml`, fixtures and any new skill. It never overwrites a file: a name clash stops the run first.
3. Validate	On errors, the model gets one repair attempt. If that fails too, the generated files are removed and the last proposal is printed.
4. Evaluate	The new agent's evals run. With `--register` and an all-pass, its name is added to the aspect's `agents` list. If any eval fails, the agent stays in the catalog but isn't added to the map.

Tips

- **Pass `--map` and `--aspect` when you know the destination.** The agent may then require only capabilities that aspect provides, the proposal won't repeat the pack's text, and the evals run with the pack composed.
- **Use `--search` to consider imported skills.** Without it, the model sees only the catalog's own skills. With it, the ten best matches for your terms are offered too. You can repeat the flag.
- **Read what it wrote.** Check the evals especially. A pass only means something if the pass conditions test what you care about, so a failure can come from the pass condition as easily as from the agent.
- **Other options.** `--model` sets the designing model (default `claude-opus-5-5`) and `--budget` caps the design call (default $1.00).

8. Write a project map

A project map tells Kitbash how a project is divided up, and which tools and agents work in each part.

Before you write it

- **Decide which pack to attach,** if any. A map may attach at most one.
- **Find the code root.** A project's code is often separate from its docs workspace and spread across several repos. List every repo and map them all.

- **Choose where the map lives.** Use `projects/<name>/` for content you can publish and `private/projects/<name>/` for everything else. Never copy private project content into a tracked path.

A working example

projects/acme/project-map.yaml

```
project: acme
catalog: ../../catalog          # relative to this file's real location, or absolute
pack: superpowers               # at most one; its base applies to every agent below
aspects:
  docs:
    paths: ["docs/**", "README.md"]
    tools: []
    agents: [docs-reviewer]
  backend:
    paths: ["services/api/**"]
    tools: [python-toolchain]
    agents: [security-reviewer, issue-triager, backend-engineer]
shared_tools: [github]          # visible in every aspect
```

Free: check one map. It may live outside this repo.

```
/usr/bin/python3 validate.py ../projects/acme/project-map.yaml
```

Bills: evaluate each agent the way this project will run it

```
/usr/bin/python3 run_evals.py docs-reviewer \
  --map ../projects/acme/project-map.yaml --aspect docs
/usr/bin/python3 run_evals.py security-reviewer \
  --map ../projects/acme/project-map.yaml --aspect backend
```

The validator doesn't check eval results

It confirms that the map is consistent, not that its agents have passed. Running the evals is your responsibility until you use `release.py`, which does enforce it.

Common edits

Swap a tool

Change one line, as long as the new tool provides the capabilities the aspect's agents require. `node-toolchain` and `python-toolchain` both provide `service-toolchain`:

```
tools: [node-toolchain]         # was [python-toolchain]
```

Leave part of the pack out of one aspect

A pack's base must suit every routed agent. If it doesn't suit one aspect, exclude that part and give the reason:

```
docs:
  paths: ["docs/**"]
  tools: []
  agents: [docs-reviewer]
  pack_exclude:
    skills: [verification-before-completion]
    reason: this agent only reads and runs no commands
```

Set a model for one aspect

```
firmware:
  paths: ["firmware/**"]
  tools: [stm32-toolchain]
  agents: [firmware-engineer]
  preferred_llm: claude-opus-5-5   # must be in the llms of every agent routed in this aspect
```

Choosing a pack

Pack	What a map's agents receive	Notes
`superpowers`	`verification-before-completion`	Evidence before any claim of success
`pstack`	`principle-prove-it-works`	A similar rule, from a set of engineering principles
`gsd`, `gstack`, `super-board`	Nothing	Their skills are commands you run, not text to compose. To use them, install them in the project itself.

9. Import skills and MCP servers

The registry holds parts imported in bulk from trusted sources. Once synced, an imported skill or server works by name, just like a catalog file.

Find and use a part

Free

```
/usr/bin/python3 registry.py sources                       # trusted sources and their sync state
/usr/bin/python3 registry.py search "pdf forms" --kind skill --limit 5
/usr/bin/python3 registry.py show pdf                      # license, resources, checkout path
```

To use a **skill**, add its name to an agent's `skills` list and rerun that agent's evals. Only the `SKILL.md` text is composed. If the skill also ships scripts or other files, the validator warns you that they won't be available. `deploy.py` warns when an agent composes a skill with a proprietary license, such as Anthropic's docx, pdf, pptx and xlsx skills, and writes it anyway.

To use an **MCP server**, bind it in a map like any other tool:

```
tools: [context7]
```

Imported servers keep secrets out of files. `registry.py show NAME` lists the environment variables a server needs, and the config refers to them as `${NAME}`. Set them before an eval or deploy. If one is missing, the validator warns you.

MCP servers you can bind today

Only servers with a capability tag can be bound. Capabilities are always tagged by hand, never inferred.

Server	Capabilities	Server	Capabilities
`github-mcp-server`	`source-control`	`terraform-mcp-server`	`infra-as-code`
`playwright-mcp`	`browser-automation`	`sentry-mcp`	`error-monitoring`
`context7`	`library-docs`	`mcp-grafana`	`observability-dashboards`
`atlassian-mcp-server`	`issue-tracking`, `wiki-docs`	`mongodb-mcp-server`	`document-database`

Server	Capabilities	Server	Capabilities
`notion-mcp`	`knowledge-base`	`mcp-redis`, `redis-mcp`	`key-value-store`
`stripe-mcp`	`payments`	`upstash-mcp-server`	`key-value-store`, `message-queue`, `vector-search`
`vercel-mcp`	`deploy-hosting`	`EnterpriseMCP`	`identity-directory`
`next-devtools-mcp`	`nextjs-devtools`	`aws-mcp`	`cloud-infra`

Add a new source

All registry configuration lives in `catalog/registry-sources.yaml`. Use this order when a project needs something that's missing:

1. **Search the registry first.** It may already be there.
2. **Vet the candidate.** Shallow-clone it and check its layout, frontmatter, that each name matches its folder, its license and any name collisions.
3. **Add it to the sources file.** Use one of the forms below.
4. **Sync:** `/usr/bin/python3 registry.py sync NAME`. Any edit to this file, including `provides` or `names`, needs a sync, or the validator reports the source as stale.

> A skill repo, pinned to a full 40-character commit

```
- name: anthropics-skills
  kind: git-skills
  url: https://github.com/anthropics/skills
  commit: ca45fc496a3436251f191354091d557da31c15b5
  layout: skills/{name}/SKILL.md
  pinned: 2026-11-02
```

> An MCP server, added to the entry that already exists for the official registry

```
- name: mcp-registry
  kind: mcp-registry
  namespaces:
    - com.notion                       # exact prefix before the '/'
  names:
    com.notion/mcp: notion-mcp         # a tool name, when the last segment would collide
  provides:
    notion-mcp: [knowledge-base]       # required before a map can bind it
```

The scripts never browse the web for parts. A map must validate and deploy from repo files and pinned sources alone. So look for new leads yourself, then add them as sources.

10. Chain agents with a playbook

A playbook runs agents in a fixed order. Code, not a model, decides whether to continue after each step, so a hard gate can't be talked around.

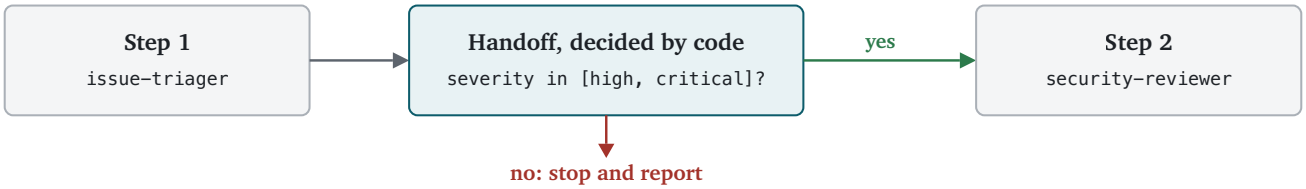


Figure 4. The steps of the sample playbook `triage-and-review`. All steps share one workspace.

Write one

A playbook lives in `playbooks/` beside its map, and its file name must match `name`. Every agent it uses must be routed in the aspect the step names.

projects/acme/playbooks/triage-then-review.yaml

```
name: triage-then-review
description: Triage an issue. If it is high or critical, security-review the change it refers to.
map: ../project-map.yaml
inputs: [issue, diff]                     # used as {issue} and {diff} below
steps:
  - agent: issue-triager
    aspect: backend
    task: "Triage {issue}"
    handoff_if: {output: severity, in: [high, critical]}
  - agent: security-reviewer
    aspect: backend
    task: "The issue in {issue} was triaged as high severity. Review {diff}."
evals:
  - inputs:
      issue: fixtures/issue-triager/issue-search-crash.md
      diff: fixtures/security/pr-sql-injection.diff
    expect_path: [issue-triager, security-reviewer]
    pass_if: "the review names the string-built query in api/users.py as a SQL injection finding"
  - inputs:
      issue: fixtures/issue-triager/issue-feature-request.md
      diff: fixtures/security/pr-sql-injection.diff
    expect_path: [issue-triager]          # proves that the gate stops the run
    pass_if: "the playbook stops after the triager; security-reviewer never runs"
```

Handoffs

Every step except the last needs exactly one `handoff_if`. A false handoff stops the run. There are no branches, retries or loops.

Form	Example	Continues when
Output	`{output: severity, in: [high, critical]}` or `{output: type, equals: bug}`	A field in the JSON block that ends the previous agent's message matches. The agent must declare the field under `outputs`. The runner asks for the block automatically.
Command	`{command: kicad.erc-gate, file: "{file}"}`	A declared command of a tool bound in that aspect, or shared, exits with status 0. `{report}` is supplied as a file in the workspace.

After the first step, each task is prefixed with the diff the previous step made, so the next agent knows what changed.

Run one

Free

```
/usr/bin/python3 validate.py ../projects/acme/project-map.yaml   # also checks its playbooks
/usr/bin/python3 run_playbook.py ../projects/acme/playbooks/triage-then-review.yaml --dry-run
```

Bills. --budget is per step, default $1.00.

```
# Run its evals. A wrong path fails without calling the judge.
/usr/bin/python3 run_playbook.py ../projects/acme/playbooks/triage-then-review.yaml

# One real run with your own inputs, and no judge
/usr/bin/python3 run_playbook.py ../projects/acme/playbooks/triage-then-review.yaml --run \
  issue=fixtures/issue-triager/issue-search-crash.md diff=fixtures/security/pr-sql-injection.diff
```

Each run also reports its handoffs, such as `handoff after issue-triager: continue (...)`, and the path it took. Add `--keep` to keep the shared workspace.

11. Deploy to a project

`deploy.py` writes a map's agents straight into a project's `.claude/` folder. It suits your own machine and maps that bind MCP tools. For anything that needs review, use a release (chapter 12).

Free: the target folder must already exist

```
/usr/bin/python3 deploy.py ../projects/acme/project-map.yaml --target ~/code/acme --dry-run
/usr/bin/python3 deploy.py ../projects/acme/project-map.yaml --target ~/code/acme
```

Output

```
wrote    .claude/agents/docs-reviewer.md
wrote    .claude/agents/security-reviewer.md
wrote    .claude/kitbash/hooks/block-network.py
wrote    .claude/agents/issue-triager.md
wrote    .claude/agents/backend-engineer.md

5 file(s) deployed to /Users/you/code/acme; manifest at .claude/kitbash.json
```

deploy.py doesn't check eval results
It requires a valid map but will deploy an agent that has never been evaluated. Make sure the map's evals have passed first.

What gets written

Each (aspect, agent) pair becomes one Claude Code subagent at `.claude/agents/<agent>.md`, or `<agent>-<aspect>.md` if the agent is routed in several aspects. Its body is built by the same function the eval runner uses. The frontmatter carries the rest:

Field	Value
`description`	The agent's description plus its aspect and paths. Claude Code reads this to decide when to use the agent.
`model`	The aspect's `preferred_llm`, otherwise the agent's.
`disallowedTools`	Tools denied by traits, plus every MCP tool in the catalog that isn't bound to this aspect.
`mcpServers`	The aspect's MCP tools, connected only while this agent runs.
`hooks`	The agent's hooks, copied to `.claude/kitbash/hooks/`.

Deploying again

The manifest `.claude/kitbash.json` records what Kitbash wrote. The next deploy overwrites those files and removes any the map no longer produces. It refuses to replace a file it didn't write:

Output

```
refusing to overwrite files Kitbash did not write (use --force):
  .claude/agents/spec-reviewer.md
```

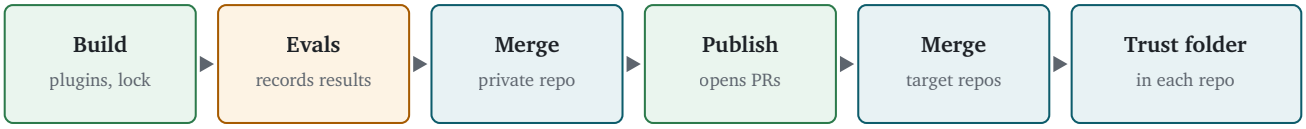
Look at that file before adding `--force`. If the map, the catalog or a playbook doesn't validate, nothing is written at all.

Use the agents

Start a new Claude Code session in the project. The subagents load from `.claude/agents/`, and Claude Code picks one when work matches its description. You can also ask for one by name.

12. Publish through a reviewed release

A release delivers agents to a project's repos as pull requests, and only after you have reviewed exactly what each agent will receive, beside its eval result.



Green: free script. Amber: bills your account. Teal: your decision. The merge in the private repo is the approval.

Figure 5. The release pipeline.

Set up a project for release

1. Create a private Git repo with a `main` branch and clone it to `private/projects/`, which this repo ignores.
2. Make sure each target repo has an `origin` with a `main` branch, and sign in to `gh`.
3. Create a project folder such as `private/projects/acme/` holding the map, any playbooks, your CLAUDE files and `release.yaml`. Point the map at the catalog with `catalog: ../../../catalog`.

private/projects/acme/release.yaml

```
target: ~/code/acme                 # the code root; may hold several repos
claude:                             # destination, relative to target or absolute: source file
  CLAUDE.md: claude/app.md
remove: [old/CLAUDE.md]             # optional: files the release should delete
eval_budget: {docs-reviewer: 2.50}  # optional: per-run budget by agent, default 1.00
```

Aspect `paths` are relative to `target` or absolute. Each aspect's paths must sit inside one Git repo, because Kitbash generates one plugin per repo.

Run a release

1. Free: generate build/. This touches no target repo.

```
/usr/bin/python3 release.py build ../private/projects/acme
```

2. Bills: run the evals of every agent without a current pass. Stops starting agents at $30 by default.

```
/usr/bin/python3 release.py evals ../private/projects/acme
/usr/bin/python3 release.py evals ../private/projects/acme \
  --only docs/docs-reviewer --max-total 5 --jobs 1
```

3. Review and approve. Commit `build/` and open a pull request in the private repo. Its diff is exactly what each agent will receive, and `build/lock.json` holds the eval results for that text. Merging is the approval. Then update your local `main`.

4. Free: check readiness, then preview

```
/usr/bin/python3 release.py check ../private/projects/acme
/usr/bin/python3 release.py publish ../private/projects/acme --dry-run
```

Output of check when the project is ready and hasn't been published

```
ready to publish

targets differ from the build:
/Users/you/code/acme
  write  .claude/skills/acme/agents/docs-reviewer.md
  write  .claude/skills/acme/.claude-plugin/plugin.json
  write  CLAUDE.md
```

`check` exits with status 1 here, because the targets still differ. That's normal before a publish. Read the words, not just the exit status.

5. Opens one PR per target repo. This is outward-facing, so run it only when you mean to.

```
/usr/bin/python3 release.py publish ../private/projects/acme
```

6. Merge the PRs in the target repos. Once they're merged, `check` reports `targets match the build`. A destination outside any Git repo, such as a docs workspace, is written directly during `publish`.

7. Trust each repo folder. Start Claude Code interactively at the root of each repo and accept the trust prompt.

> **A plugin can fail to load without any warning**
> A repo plugin loads only when a session starts at the repo root, not in a subfolder, and only once that exact folder is trusted. `claude -p` or a trusted parent folder isn't enough, and `publish` never sets trust for you. Plugin agents are named `<plugin>:<agent>`.

What makes publish refuse

Message	What to do
`... has uncommitted changes`	Commit or discard your changes in the private repo.
`... is not level with origin/main`	Merge your PR and pull. Unmerged work hasn't been approved.
`the committed build differs from a fresh build`	Something changed after the build. Rerun `build`, then review and merge the result.
`ASPECT/AGENT: the agent changed since the last build`	A change altered the agent's text, hooks or evals, which voids its recorded pass. Run `build` and `evals`.
`ASPECT/AGENT: no eval result`	Run `release.py evals`.
`ASPECT/AGENT: evals passed N of M`	Fix the failure and run `evals` again.

Limits of a release

- **Tool scope is per repo,** not per agent as with `deploy.py`.
- **A map that binds an MCP tool is refused:** `a plugin cannot bind MCP tools yet`. Use `deploy.py` for it.
- **Fixtures aren't in the hash.** Changing a fixture doesn't void a recorded pass, so rerun those evals yourself.
- **Publishing is yours to approve.** If an assistant runs Kitbash for you, don't let it merge these PRs or run `publish` unless you ask.

13. What's in the catalog

This chapter lists the catalog as of this guide's date. To see what a map actually routes, read the map.

Agents

Agent	Does	Its evals need
`security-reviewer`	Reviews a PR or file set for security defects only. Read-only.	Nothing extra
`spec-reviewer`	Finds statements in specification documents that contradict each other, citing both sides. Read-only.	Nothing extra
`issue-triager`	Labels issues by type and severity and drafts a first reply. Declares `type` and `severity` outputs.	Nothing extra
`backend-engineer`	Implements and fixes TypeScript and Python services, testing first.	Python; Node and `pnpm`. Run through a map.
`mobile-engineer`	Implements and fixes Kotlin and SwiftUI companion apps.	Xcode for Swift. Run through a map.
`firmware-engineer`	Implements and fixes STM32 firmware, building and testing first.	An `arm-none-eabi` toolchain with newlib
`drc-checker`	Runs ERC and DRC on KiCad files and reports every violation. Read-only.	KiCad
`schematic-engineer`	Creates and modifies KiCad schematics, proving each change with ERC and a netlist.	KiCad and `kicad-sch-api`
`ux-designer`	Creates and revises screens in a Pen design file, reusing tokens.	Pen desktop app, running
`mechanical-engineer`	Revises parametric FreeCAD parts through a Params sheet. Its evals haven't been run, so treat it as unproven.	FreeCAD with the `freecad-mcp` addon, and an aspect that binds `freecad`
`recipe-author`	Turns videos, pages or text into validated recipes for the Epicura cooking device.	`yt-dlp` and an Epicura recipe engine checkout. YouTube may rate-limit caption fetches.

Traits

Trait	Effect
`read-only`	Inspect, never modify. Denies `Edit` and `Write`.
`runs-tests`	Run the project's tests before reporting a change as done, and show the result.
`concise-reporter`	Lead with the conclusion, one sentence per finding, cite `path:line`.

Tools and capabilities

Tool	Kind	Provides	Commands
`github`	cli	`source-control`	`pr_diff`, `pr_comment`
`python-toolchain`	cli	`service-toolchain`	`test`, `pytest`
`node-toolchain`	cli	`service-toolchain`	`install`, `test`, `lint`, `build`
`xcode-toolchain`	cli	`mobile-toolchain`	`test`, `package_test`, `build`
`android-toolchain`	cli	`mobile-toolchain`	`build`, `test`, `lint`
`stm32-toolchain`	cli	`embedded-toolchain`	`build`, `flash`, `test`
`kicad`	cli	`schematic-erc`, `design-rule-check`	`cli`, `erc`, `drc`, `erc-gate`, `drc-gate`

Tool	Kind	Provides	Commands
`kicad-sch-api`	cli	`schematic-capture`	`python`
`recipe-validator`	cli	`recipe-validation`	`validate`, `validate-gate`
`video-transcript`	cli	`video-transcript`	`transcript`
`pencil`	mcp	`design-canvas`, `design-tokens`	Through its MCP server
`freecad`	mcp	`mechanical-cad`	Through its MCP server

Skills and hooks

Skills: `backend-service`, `issue-triage`, `kicad-eda`, `kicad-schematic`, `mechanical-design`, `mobile-app`, `pencil-design`, `recipe-authoring`, `security-review`, `spec-consistency`, `stm32-firmware`. The registry adds 40 imported skills.

Hooks: `block-network.py` blocks shell commands such as `curl`, `ssh` and `npm install`. `only-for-agents.py` limits a plugin's hook to named agents. Releases add it for you.

14. Command reference

Run everything from `catalog/`. Every script except `validate.py` also has `--help`. `validate.py` has no help option and simply runs the checks.

`validate.py` FREE

`validate.py`	Catalog, plus maps in `../projects/` and `../private/projects/`
`validate.py PATH/TO/project-map.yaml ...`	The named maps, which may live anywhere, and their playbooks
`validate.py --catalog DIR`	Catalog only

Exits 1 on any error. Warnings don't fail the run.

`run_evals.py AGENT ...` BILLS

`--all`	Every agent. Costly, and it needs every external tool.
`--map FILE --aspect NAME`	Compose with the map's pack and the aspect's tools
`--dry-run`	Print the composed prompt and commands. Free.
`--budget USD`	Cap per agent run, default 1.00. The judge gets 0.20.
`--max-turns N`	Turn limit per agent run, default 30
`--judge-model MODEL`	Model that grades, default `claude-sonnet-5-5`
`--keep`, `--workdir DIR`	Keep workspaces, or put them somewhere other than `catalog/.evals/`
`--report FILE`	Also write results as JSON

`run_playbook.py PLAYBOOK` BILLS

`--run KEY=VALUE ...`	One run with these inputs, without a judge, instead of the evals
`--dry-run`	Print each step and handoff. Free.
`--budget USD`, `--max-turns N`	Per step: default 1.00 and 30
`--judge-model`, `--keep`, `--workdir`, `--report`	As for `run_evals.py`

`make_agent.py "REQUEST"` BILLS, EVEN WITH --dry-run

`--map FILE --aspect NAME`	Limit capabilities to that aspect and evaluate through the map
`--register`	With a map, add the agent to the aspect on an all-pass
`--search TERMS`	Offer the best-matching imported skills. Repeatable.
`--no-evals`	Stop after validation
`--dry-run`	Print the proposal and write nothing
`--model MODEL`, `--budget USD`	Designing model, default `claude-opus-5-5`; design-call cap, default 1.00

`registry.py` FREE

`registry.py sources`	Trusted sources and their sync state
`registry.py sync [NAME ...]`	Fetch every source, or the named ones. Uses the network.
`registry.py search QUERY [--kind skill|mcp] [--limit N] [--json]`	Search imported skills and servers
`registry.py show NAME`	License, resources and checkout path. For a server: config and secret names.

`deploy.py MAP --target DIR` FREE

`--dry-run`	Print the files and write nothing
`--force`	Overwrite agent files Kitbash didn't write

`release.py`

`build PROJECT` FREE	Regenerate `build/`. Touches no target repo.
`evals PROJECT` BILLS	Run the evals of every agent without a current pass
`--only ASPECT/AGENT ...`	Limit to these
`--max-total USD`, `--jobs N`, `--workdir DIR`	Ceiling, default 30; parallel agents, default 3; workspace, default a temp folder
`check PROJECT` FREE	Readiness, and drift against each target's `origin/main`. Exits 1 if not ready or if targets differ.
`publish PROJECT` FREE	Open a PR in each target repo that differs. Outward-facing.
`--dry-run`	List the changes. Refuses if the project isn't ready.
`--trailer TEXT`, `--pr-footer TEXT`	Last lines of each commit message and PR description

15. File formats

Every file is YAML, with a Markdown body for traits, skills and packs. A file's `name` must match its file name or folder. **Bold** fields are required. Three small conventions appear inside values: `{name}` placeholders, `tool.command` references, and list order meaning merge order.

Agent: `agents/<name>.yaml`

`name`, `description`	Identity. Deploys use the description for routing, so always write one.
`traits`, `skills`	Lists, composed in order
`llms`, **`preferred_llm`**	Allowed models, and the default, which must be in the list
`prompt`	The agent's own text
`evals`	List of `task`, `pass_if`, and optionally `creates`, the new files a task makes. Any `fixtures/...` path in a task must exist unless it's listed under `creates`, in which case it must not exist yet.
`requires.capabilities`	What the aspect's tools must provide
`hooks`	`{pre_tool_use: [hooks/NAME.py]}`
`outputs`	`{field: one-line description}`, read by playbook handoffs
`overrides`	`{allow: [TOOL], reason: "..."}`

Trait: `traits/<name>.md`

Frontmatter has **`name`**, `tools` and `denies`. A single trait can't both use and deny a tool. The Markdown body is the composed text.

Skill: `skills/<name>/SKILL.md`

Frontmatter has **`name`**, **`description`** and `requires`, which may hold `tools`, `capabilities` and `min_model_tier` (`haiku`, `sonnet`, `opus` or `fable`). The Markdown body is the composed text.

Tool: `tools/cli/<name>.yaml` or `tools/mcp/<name>.yaml`

`name`, **`kind`**, `description`	`kind` is `cli` or `mcp` and must match the folder
`provides`	At least one capability
`commands`	Named shell commands, with `{placeholder}` arguments
`server`	For MCP: `command` and `args`
`skill`	Path to a skill that teaches the tool, or `null`
`artifacts`	Globs of side-effect files to hide from the judge's diff
`inspect`	For document fixtures such as `.pen`: `extensions`, `open` and `dump` commands

Pack: `packs/<name>.md`

Frontmatter has `name`, `description`, `source` (**`url`**, a full 40-character **`commit`**, `version`, `license`), `layout` (such as `skills: skills/{name}/SKILL.md`) and `base` (`traits`, `skills`).

Project map: `project-map.yaml`

`project`, **`catalog`**	Project name, and the catalog path: relative to the map's real location, or absolute. `~` is expanded and symbolic links are followed.
`pack`	One pack name
`shared_tools`	Tools visible in every aspect
`aspects.<name>`	**`paths`** (globs), `tools`, `agents`, `preferred_llm`, `pack_exclude` (`traits`, `skills`, **`reason`**)

Playbook: `playbooks/<name>.yaml`

`name`, `description`	Identity
`map`	Must name the map beside this `playbooks/` folder
`inputs`	Names usable as `{placeholder}` in tasks and handoffs
`steps`	List of **`agent`**, **`aspect`**, **`task`** and `handoff_if`, which every step but the last must have
`evals`	List of **`inputs`** covering every declared input, **`expect_path`** (the agents you expect to run, a prefix of the steps) and **`pass_if`**

Release settings: `release.yaml`

`target` is the code root. `claude` maps each destination to a source file. `remove` lists files to delete. `eval_budget` sets a per-run budget by agent.

16. Validator rules

The validator runs automatically before evals, playbooks and deploys. These are the checks it makes.

Rule	What must be true
1	`preferred_llm` is in the agent's `llms`. An aspect's `preferred_llm` is in every routed agent's `llms`.
2	A skill's required tools aren't denied by a trait, and the agent's `preferred_llm` meets the skill's `min_model_tier`.
3	An aspect's tools plus `shared_tools` provide every capability its agents require.
4	A tool that one trait uses and another denies is listed under `overrides.allow` with a `reason`.
5	`pack` names exactly one existing pack, and its base can be satisfied by every routed agent. If it can't, the aspect has a `pack_exclude` with a reason.
6	A playbook names its map; each step's agent is routed in its aspect; each step but the last has one handoff a bound tool command or a declared output can decide; every placeholder is a declared input; and evals exist with a valid `expect_path`.
R1–R3	`registry-sources.yaml` is well formed; no source has changed since its last sync; imported names don't collide with catalog files or each other.
R4	A map binds an imported MCP server only if the sources file tags it under `provides`.
R5–R7	*Warnings:* a bound server's secret variable is unset; a composed imported skill ships files that won't be available; the registry hasn't been synced.

It also checks that names match file names, no name is used twice, and every referenced trait, skill, hook, fixture, tool and agent exists. Non-executable hooks and overrides that allow nothing denied produce warnings.

17. Troubleshooting

Validator errors

Message	Fix
`agent 'X' requires ['source-control']; bound tools provide ['service-toolchain']`	Bind a tool that provides the capability in that aspect or in `shared_tools`.
`aspect preferred_llm 'M' is not in agent 'X' llms`	Remove or change the aspect's model, add the model to the agent's `llms` and rerun its evals, or move the agent to another aspect.
`preferred_llm 'M' is not in llms [...]`	Add it to `llms` or choose a listed model.
`skill 'S' needs min_model_tier sonnet; preferred_llm 'M' is below it`	Use a stronger `preferred_llm`.
`traits conflict on ['Edit']; add overrides.allow with a reason`	Remove the conflicting trait, or add `overrides` with `allow` and `reason`.
`name 'X' does not match path (expected 'Y')`	Make `name` and the file name agree.
`unknown trait 'X'`, `unknown agent 'X'`, `unknown tool 'X'`	Check the spelling. For an import, run `registry.py sync`.
`evals[N] references missing fixture fixtures/...`	Create the fixture, or list the path under `creates:` if the task makes it.
`evals[N] says it creates ..., but that fixture already exists`	Delete the fixture or use a new path.
`'pack' must name exactly one pack, not a list`	Keep one pack.
`agent 'X' is not routed in aspect 'A'`	Add the agent to that aspect's `agents`, after its evals pass.
`output handoff on 'F', which agent 'X' does not declare under 'outputs'`	Declare the field in the agent, then rerun its evals, or use a declared field.
`expect_path [...] is not a non-empty prefix of the step agents`	List the agents from the first step onward, stopping where the run should stop.
`task uses undeclared input(s) ['X']`	Add it to `inputs` or fix the placeholder.
`registry is stale for source 'S'; run registry.py sync S`	Run that command.
`imported server 'S' provides no capabilities`	Tag it under `provides` in the sources file, then sync.

Other problems

Symptom	Cause and fix
`ModuleNotFoundError: No module named 'yaml'`	The wrong interpreter. Use `/usr/bin/python3`, or install PyYAML for yours.
`no registry at ...; run registry.py sync`	The registry hasn't been built. Run `registry.py sync`.
`target is not a directory`	Create the `--target` folder first.
`a plugin needs each aspect in one git repo`	An aspect's paths cross repos or aren't in one. Split it into one aspect per repo.
An imported MCP server won't connect	Set the variables listed by `registry.py show NAME`.
A released plugin's agents don't appear	Start the session at the repo root, and trust that exact folder. Nothing warns you about this.
The judge fails an eval on a huge or irrelevant diff	Run through a map whose tool declares `artifacts`, or add `artifacts` to the tool.

Symptom	Cause and fix
A `command` handoff always passes	The tool exits 0 even on failure. Declare a gate command that exits nonzero, as `kicad.erc-gate` does.
An eval hits its budget or turn limit	Raise `--budget` or `--max-turns`, or set `eval_budget` for releases.
A skill or a tool's command can't be found on your machine	Its file has a path from another machine. Edit the tool or skill file.
A recipe eval reports HTTP 429	YouTube rate-limits caption fetches. Those evals accept an honest failure report.

18. Cheat sheet

Run from `catalog/`. `py` here means `/usr/bin/python3`. Green is free, amber bills.

Check

```
py validate.py
py validate.py ../projects/NAME/project-map.yaml
```

Look inside

```
py run_evals.py AGENT --dry-run
py run_evals.py AGENT --dry-run \
   --map MAP --aspect ASPECT
py run_playbook.py PLAYBOOK --dry-run
py deploy.py MAP --target DIR --dry-run
```

Evaluate

```
py run_evals.py AGENT
py run_evals.py AGENT --map MAP --aspect ASPECT
py run_evals.py AGENT --budget 0.50 --keep
py run_playbook.py PLAYBOOK
```

Create an agent

```
py make_agent.py "REQUEST" --dry-run
py make_agent.py "REQUEST" --no-evals
py make_agent.py "REQUEST" --search "TERMS"
py make_agent.py "REQUEST" \
   --map MAP --aspect ASPECT --register
```

Find and import parts

```
py registry.py sources
py registry.py search "TERMS" --kind skill
py registry.py show NAME
py registry.py sync [SOURCE]
```

Run a playbook once

```
py run_playbook.py PLAYBOOK --run KEY=VALUE ...
```

Deploy

```
py deploy.py MAP --target DIR
py deploy.py MAP --target DIR --force
```

Release

```
py release.py build PROJECT
```

```
py release.py evals PROJECT
```

```
# review and merge in the private repo
```

```
py release.py check PROJECT
py release.py publish PROJECT --dry-run
py release.py publish PROJECT
```

```
# merge the PRs, then trust each repo folder
```

Remember

- **Evals gate registration.** Don't route an agent until its evals pass, and rerun them after changing anything it composes.
- **Changing a map's pack** means rerunning the evals of every agent in that map.
- **After editing `registry-sources.yaml`**, run `registry.py sync`.
- **A clean validator run** ends with 0 errors and 3 warnings.
- **Only `release.py` enforces eval results.** The validator and `deploy.py` don't check them.